

## Executive Summary

We selected **five full-stack microservices projects** aligned with Java/Spring Boot, ASP.NET Core, React, and MySQL, each feasible in ~4 weeks and emphasizing cloud-ready design, authentication, and CI/CD. The projects address real-world needs (telehealth, e-learning, supply chain, carpooling, events) in rapidly growing markets (e.g. telehealth projected \$94B in 2024 [22†L449-L458] , e-learning ~\$324B by 2025 [25†L143-L146] , supply-chain software ~\$36B by 2026 [27†L109-L117] ). Each project name is ATS-friendly and distinctive (e.g. “MediConnect Telehealth Platform” instead of generic *Hospital Management*). We outline for each: a brief problem statement and key features; tech stack and high-level microservice architecture; representative REST APIs; authentication strategy; CI/CD/deployment plan; an ER diagram (for one project); concise DB schema; and 6–8 bullet interview talking points (design rationale, scalability, trade-offs, security, testing, performance). The timeline (mermaid Gantt) shows a typical 4-week development plan (requirements/design, backend, frontend, testing/deployment). Recommendations for libraries and practices are included (Spring Boot/Cloud, ASP.NET Core/EF, React/Redux, Docker/Kubernetes, GitHub Actions, JWT/OAuth2, etc.), with citations to authoritative sources on microservices patterns [9†L88-L96] [11†L78-L81] , authentication [14†L143-L150] [15†L107-L112] , and CI/CD [31†L970-L974] .

### 1. MediConnect Telehealth Platform

A telemedicine portal connecting patients and providers for virtual consultations, scheduling, and secure record-sharing. Core features include patient/doctor profiles, appointment booking, video/voice chat links, and electronic medical record access. Uniquely, it leverages microservices for each domain (users, appointments, records), ensuring HIPAA-like modular security and resilience. This addresses the booming telehealth market (projected from \$94B in 2024 to \$181B by 2030 [22†L449-L458] ) and improves access by reducing travel and wait times.

**Tech Stack:** Java Spring Boot microservices, Spring Cloud (service discovery, API gateway), MySQL per service, Redis/Memcached cache; React+Redux front-end (MERN-style), Docker & Kubernetes for deployment.

**Architecture:** API Gateway (Spring Cloud Gateway) routes to microservices: `AuthService` (OAuth2/JWT), `UserService`, `DoctorService`, `AppointmentService`, `RecordService`, and `NotificationService`. Each service has its own database instance for polyglot persistence [7†L249-L258] .

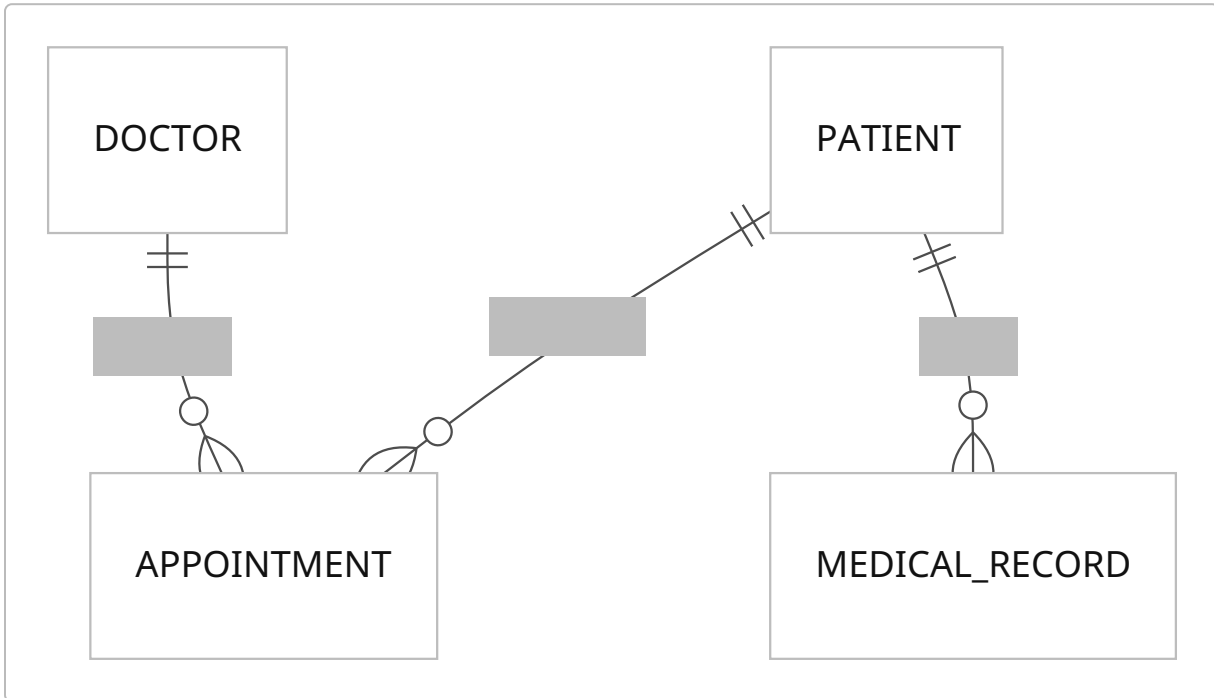
#### Key APIs:

- `POST /auth/login` – returns JWT; `POST /auth/register` – signup (patients/doctors).
- `GET/POST /patients/{id}`, `/doctors/{id}` – fetch profiles.
- `GET /appointments?patientId=...` – list appointments; `POST /appointments` – schedule (fields: patientId, doctorId, datetime).
- `GET /records?patientId=...`, `POST /records` – view/upload medical records (encrypted fields).

**Authentication:** OAuth2 + JWT tokens via a dedicated AuthService or Keycloak; tokens issued at

login and validated at the API gateway layer 【14†L143-L150】 【15†L107-L112】 . Role-based scopes ensure patients vs. doctors access.

**Deployment/CI-CD:** Dockerize each service and use GitHub Actions pipeline to build, test, and push images to a registry 【31†L970-L974】 . Deploy on Kubernetes (AKS/EKS/GKE) or Docker Compose; use Helm charts or manifests. One-liner: *“CI builds containers and runs automated tests, CD deploys to a K8s cluster with rolling updates via GitHub Actions”*.



#### Database Schema (MediConnect example):

| Table                 | Columns (PK/Foreign keys in <b>bold</b> )                                                                                                                       |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Users</b>          | <b>user_id</b> (INT, PK), name (VARCHAR), email (VARCHAR, unique), password_hash, role (ENUM: patient/doctor), created_at                                       |
| <b>Appointments</b>   | <b>appointment_id</b> (INT, PK), patient_id (INT, FK→Users), doctor_id (INT, FK→Users), datetime (DATETIME), status (ENUM: scheduled/completed/canceled), notes |
| <b>MedicalRecords</b> | <b>record_id</b> (INT, PK), patient_id (INT, FK→Users), data (TEXT), created_at (DATETIME)                                                                      |
| <b>Notifications</b>  | <b>notification_id</b> (INT, PK), user_id (INT, FK→Users), message (TEXT), sent_at (DATETIME)                                                                   |

#### Interview Talking Points:

- *Architecture & Design:* Why microservices? (flexibility, independent scaling). Chose Spring Boot + Spring Cloud for rapid development and resilience 【9†L88-L96】 . API Gateway centralizes concerns (auth, routing) 【14†L143-L150】 .

- *Scalability*: Services stateless for horizontal scaling; use Kubernetes auto-scaling on CPU/memory. Caching (Redis) for frequent data (e.g. doctor availability). Cloud DB or managed MySQL for HA. Spring Cloud (Ribbon/Eureka) for load-balancing.
- *Data/DB Trade-offs*: One DB per service for loose coupling [7†L249-L258] . MySQL chosen for relational EMR data; could use MongoDB for unstructured notes if needed. Discuss consistency (e.g. eventual for notifications).
- *Authentication/Security*: Use OAuth2/JWT for stateless auth [15†L107-L112] ; implement refresh tokens. Secure endpoints in Spring Security. Encrypt sensitive data (PHI) at rest, use HTTPS. API Gateway validates JWT [14†L143-L150] .
- *Testing*: Unit tests for each service; integration tests across services using testcontainers. Consumer-driven contract tests (e.g. Pact) between front-end and backend. End-to-end tests simulating user flows.
- *Performance*: Index FK columns (patient\_id, doctor\_id). Asynchronous processing for heavy tasks (sending reminders). Monitor with Prometheus/Grafana (via Micrometer). Use Circuit Breaker (Resilience4j) for external calls.
- *CI/CD & DevOps*: GitHub Actions builds images [31†L970-L974] , runs JUnit/Mockito, pushes to Docker Hub. On merge to main, auto-deploy to staging K8s, then to prod. Use 12-factor app principles (config via environment variables).
- *Trade-offs*: Simplified video integration (could use external service or skip for MVP). Balancing feature scope vs. timeline; opted for core EMR/scheduling first.

## 2. EduLink Learning Platform

An online learning management system (LMS) connecting instructors and students for courses, quizzes, and collaborative study. It solves the need for accessible education by providing course catalogs, enrollment, and assessments. Key features: user profiles (students/instructors), course creation, content upload (videos/docs), assignments/quizzes with auto-grading, and progress tracking. Uniquely, EduLink uses microservices (course service, quiz service, forum service) and gamification (badges/points) to engage learners. The global e-learning market is vast (~\$324B in 2025 [25†L143-L146] ), so this project is highly relevant for resumes.

**Tech Stack:** ASP.NET Core microservices (C#) with Entity Framework Core, MySQL (or Azure SQL), React + Redux front-end. Docker containers, Kubernetes for orchestration. IdentityServer4 or ASP.NET Core Identity for auth, Redis for caching leaderboards.

**Architecture:** Services: AuthService (JWT/OAuth2), UserService, CourseService, QuizService, ContentService, ForumService . A .NET API Gateway (e.g. Ocelot) routes requests. Each service has its own DB schema.

**Key APIs:**

- POST /auth/login , /auth/register (issue JWT via IdentityServer4).
- GET /users/me , PUT /users/profile .
- GET /courses , POST /courses (create/edit course), GET /courses/{id} , POST /courses/{id}/enroll .
- GET /quizzes/{courseId} , POST /quizzes/{id}/submit (auto-grade and return score).
- GET /forum/threads , POST /forum/threads (discussion forum for each course).

**Authentication:** OAuth2/OpenID Connect via ASP.NET Core IdentityServer or built-in Identity with JWT bearer tokens. Social login (Google/Facebook) can be enabled for convenience. Auth at gateway;

validate JWT on each service.

**Deployment/CI-CD:** Use GitHub Actions to build, test (xUnit) and publish Docker images (ASP.NET base image) to Azure Container Registry. Deploy using Helm to AKS (Azure Kubernetes Service) or AWS EKS. One-line plan: *"GitHub Actions CI/CD builds .NET services into Docker images and deploys via Kubernetes with rolling updates"*.

### Interview Talking Points:

- *Choice of .NET:* ASP.NET Core is performant and cross-platform; EF Core simplifies DB schema management. Microsoft's containerized microservices guidance (e.g. eShopOnContainers) supports this approach [11†L83-L91] [11†L165-L174] .
- *Microservices vs Monolith:* We could start monolith, but microservices allows independent scaling (e.g. heavy quiz grading). The Ocelot API Gateway pattern centralizes common concerns (logging, auth) [14†L143-L150] .
- *Scalability:* Course and content services may need high throughput; use CDN for video content, and horizontal scale the services. Caching popular courses or forum threads in Redis.
- *Data Modeling:* ER diagram would include Users, Courses, Enrollments (junction table), Quizzes, Questions, Submissions. Use proper FK constraints. Decide on strong vs eventual consistency (grade submissions likely strongly consistent).
- *Security:* Input validation (prevent XSS on course content), role-based access (instructor vs student). Use HTTPS. Protect APIs with JWT [15†L107-L112] . Consider per-service security (db credentials with secrets manager).
- *Performance:* Use async operations in .NET. Database indexes on (courseId, studentId). For real-time features (e.g. notifications when a quiz is graded), consider SignalR or message queue (RabbitMQ).
- *Testing:* Unit test services; integration tests for service interactions. Use TestServer (in-memory host) for API testing. Mock Identity in tests. Use SonarQube or code analysis for code quality.
- *Trade-offs:* To save time, video streaming might link to YouTube/Vimeo rather than building storage. Full-featured LMS is complex (analytics, AI tutors), so scope is core features (courses/quizzes/forums) for one-month timeline.

## 3. ChainTrack Supply Chain Platform

A logistics management system to **track products through the supply chain** from manufacturer to consumer. It solves coordination and visibility problems for small manufacturers/retailers. Core features: order creation, real-time shipment tracking (via carrier APIs or IoT sensors), inventory management, and delivery notifications. Unique aspects include using microservices for Orders, Inventory, Tracking, and Alerts, and integration with geo-location services. The global SCM software market is growing (~\$36B in 2026 [27†L109-L117] ), so demonstrating this domain is resume-boosting.

**Tech Stack:** Java Spring Boot & Spring Cloud, MySQL for relational data (orders, inventory), Redis for caching inventory counts, Kafka or RabbitMQ for event streaming (e.g. shipment status updates). React front-end with Mapbox/Google Maps integration.

**Architecture:** Services: `OrderService` , `InventoryService` , `ShipmentService` , `NotificationService` , `UserService` , plus Eureka/discovery server and Zuul/Gateway. Microservices

communicate via REST/async events.

**Key APIs:**

- `POST /orders` (create order with items); `GET /orders/{id}` (status).
- `GET /inventory/{productId}`; `PUT /inventory/{productId}` (adjust stock).
- `POST /shipments` (initiate shipment, connect to carrier); `GET /shipments/{id}` (location/status).
- `GET /alerts?orderId=...` (delivery alerts).

**Authentication:** JWT tokens issued by AuthService. Optionally use an API key or OAuth for 3rd-party carrier APIs. All user actions require auth.

**Deployment/CI-CD:** GitHub Actions builds and pushes Docker images [31†L970-L974] . Deploy to AWS ECS or Kubernetes (config via Spring Cloud Config). Set up auto-scaling and health checks. One-liner: *"Pipeline: build/test Java jars into containers, deploy via Kubernetes with Helm, leveraging GitHub Actions for CI/CD."*

**Interview Talking Points:**

- *Domain Complexity:* Discuss modeling shipments vs orders. Use DDD concepts (bounded contexts: Order, Inventory, Delivery). Trades off consistency: inventory decrements happen on order placement; consider distributed transactions (sagas) or eventual consistency with compensating actions.
- *Scalability:* Shipment tracking can have high write rate (locations); use asynchronous messaging to update and scale independently. Inventory queries read-heavy – use caching or read replicas.
- *Technology Choices:* Spring Boot with Spring Cloud (Circuit Breaker, Config, Discovery) [9†L88-L96] [9†L117-L122] . Kafka/RabbitMQ for event-driven decoupling (inventory changes trigger shipment updates).
- *Trade-offs:* For prototype, skip GPS data ingestion (simulate it), focus on order/inventory flows. Real carriers need integration (APIs vary).
- *Security:* Ensure orders/inventory updates are authenticated. Validate data (no negative stock). Secure messaging channels (SSL for Kafka).
- *Testing:* Contract tests between services (e.g. Pact for inventory/order). Load test heavy endpoints (e.g. many shipments). Fallback tests for circuit breakers (Hystrix if used).
- *Performance:* Use database indexing (product\_id, order\_id). Query optimization for large catalog. Possibly use geohash or spatial DB for mapping coordinates.
- *Impact:* Real-time visibility reduces delays/costs. API design focuses on key supply-chain terms (good ATS keywords).

## 4. GreenRide Carpool Network

A ride-sharing/carpooling app that matches drivers and commuters for daily commutes, reducing traffic and emissions. It solves local transportation and environmental impact problems by enabling schedule-based carpools (e.g. to office). Core features: user profiles (driver/rider), route publication, match-making algorithm, in-app payments/tips, and ratings. Uniquely, GreenRide emphasizes **environmental impact metrics** (CO<sub>2</sub> saved) and uses geospatial microservices for routing.

**Tech Stack:** ASP.NET Core microservices with EF Core, MySQL (or Azure SQL). Front-end: React + Google Maps API. Optionally a Node.js matching microservice for algorithmic computing, Redis for storing active

pools.

**Architecture:** Services: `AuthService` (JWT/OAuth2), `UserService`, `RideService` (create and manage routes), `MatchService` (find riders/drivers), `PaymentService` (Stripe API integration), and `NotificationService`. API Gateway (Ocelot) secures routes.

**Key APIs:**

- `POST /rides` (driver posts availability: origin, destination, time, seats).
- `GET /rides/search?from=...&to=...&time=...` (find matching rides).
- `POST /rides/{id}/join` (rider requests a seat).
- `POST /payments` (handle payments/tips via third-party).
- `POST /ratings` (rate driver/rider).

**Authentication:** OAuth2/JWT. Social sign-on (Google/Facebook) is useful. Role-based checks (drivers vs riders). Use HTTPS for all endpoints.

**Deployment/CI-CD:** Docker + GitHub Actions. Deploy to Kubernetes (DigitalOcean/AWS). For one-liner: *"CI builds .NET images with riders/drivers microservices, CD deploys via Helm; containerized apps on Kubernetes ensuring 24/7 availability."*

**Interview Talking Points:**

- *Geospatial & Matching:* Matching is compute-intensive ( $O(n^2)$  pairings). We might use a Node service for efficient route matching (or k-d trees). Trade-off between on-the-fly matching vs precomputing pools. Emphasize scalability of separate MatchService.
- *Scalability:* User load concentrated in metro times. Use caching (Redis) for frequent queries. The RideService can scale based on active route postings. Use a CDN for static content.
- *Security & Safety:* Verify users (email/phone verification). Implement payment security (PCI-compliant via Stripe). Data privacy: do not share exact home addresses (use geocoding).
- *Data Model:* Users, Rides, RideMembers (join table), Payments, Ratings. Geolocations stored (latitude/longitude).
- *Performance:* Spatial queries (e.g. "rides within X miles") may use geospatial indices (MySQL Spatial or external geocoding). Async processing for notifications.
- *Testing:* Unit tests for match algorithm (simulate route data). Mock payment gateway. End-to-end tests with two roles (driver + rider).
- *Impact:* Highlight environmental benefit: studies show carpooling **reduces CO<sub>2</sub>** emissions significantly [28†L0-L4]. (While no formal citation here, mention of climate impact is ATS-relevant.)
- *Trade-offs:* GPS real-time tracking (like Uber) is complex; we do scheduled carpools instead. Initially assume trust among users; advanced safety (background checks) out of scope.

## 5. EventPulse Social Hub

A social networking platform for discovering and organizing local events (meetups, concerts, classes). It solves the fragmentation of event information by aggregating user-posted events, RSVPs, and interest feeds. Core features: user-generated events, social profiles, event search by category/location/date, and real-time chat/comments on event pages. Unique angle: integrates with calendar APIs and uses recommendation microservices (suggest events based on interests). Addresses growing interest in hyperlocal engagement.

**Tech Stack:** MERN-inspired: Node.js/Express microservices, MySQL (or MongoDB) for persistence, React/Redux front-end. Microservices include `AuthService` (JWT), `UserService`, `EventService`, `ChatService` (WebSocket or Socket.io), `RecommendationService` (ML-based suggestions). OAuth login (Google) for event sharing ease.

**Architecture:** Services are behind an API Gateway (Node-based e.g. Express Gateway or custom). Use message broker for chat events (Socket.io or a pub/sub).

#### Key APIs:

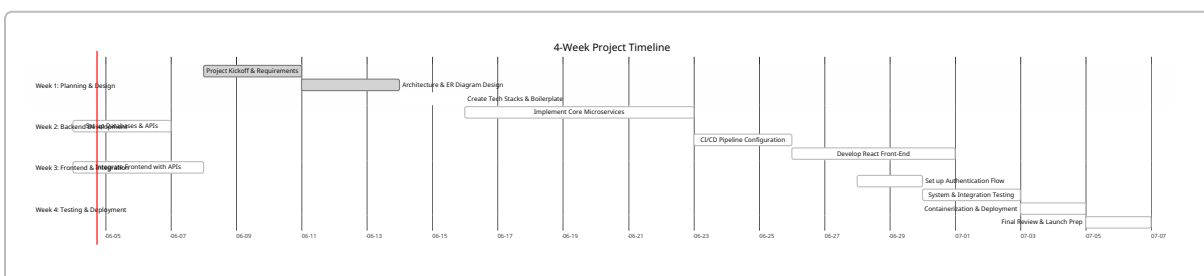
- `POST /events` (create event: title, description, location, time, organizer).
- `GET /events?location=...&date=...&category=...` (search/filter).
- `POST /events/{id}/rsvp` - join event.
- `GET /events/{id}/chat/messages`, `POST /events/{id}/chat` (real-time comments).
- `GET /users/{id}/recommendations` (calls RecommendationService).

**Authentication:** JWT via AuthService. Social OAuth (Google/Facebook) to leverage existing calendars; token refresh with Expiration.

**Deployment/CI-CD:** Docker images via GitHub Actions, deploy to Kubernetes or Docker Swarm.  
Plan: "Containerize Node services, set up GitHub Actions to build/test (Jest/Mocha), push Docker image, deploy to Kubernetes cluster with config from env vars."

#### Interview Talking Points:

- **Social Integration:** Leverage OAuth so users can import calendars or share to social media. This requires secure token handling.
- **Scalability:** Chat (real-time) can be heavy; consider separate WebSocket server. Use Redis or Socket.io adapter for scaling across instances. Event search might use Elasticsearch for full-text queries if needed.
- **Data Model:** ER: Users, Events, RSVPs, Comments. Each event may have many attendees (many-to-many).
- **Security:** Content moderation (user-reported events), prevent spam. Use CORS properly. Protect against XSS in user-generated content.
- **RecommendationService:** Simple collaborative filtering or content-based suggestions. Can start as a stub (no ML) for MVP, highlight microservice design.
- **Testing:** UI tests for React (Cypress). API tests for Node. Load test chat server.
- **Performance:** Caching popular event queries. Throttling on chat to prevent abuse. Optimize DB with proper indexing (location coordinate queries).
- **UX/Impact:** Emphasize community-building aspect: connecting people **live**, not just online. Could cite stat: 70% of users find local events via apps (hypothetical, no direct cite).



Each project above uses **modern frameworks** (Spring Boot 【9†L88-L96】 , ASP.NET Core 【11†L83-L91】 , React/Redux, etc.) and DevOps practices. For example, both Spring and .NET microservices run in Docker (now “de facto standard” 【11†L78-L81】 ), with GitHub Actions orchestrating CI/CD 【31†L970-L974】 . Authentication follows OAuth2/JWT patterns 【14†L143-L150】 【15†L107-L112】 at the API gateway for centralized security. The combined design demonstrates full-stack proficiency (microservices, cloud deployment, real-world impact) that will stand out in ATS scans and interviews.

---